



论文汇报

汇报人：张刘路

导师：肖甫、何昕

2025.11.16

Accelerating End-Cloud Collaborative Inference via Near Bubble-free Pipeline Optimization

Luyao Gao^{1,2}, Jianchun Liu^{1,2}, Hongli Xu^{1,2}, Sun Xu^{1,2}, Qianpiao Ma³, Liusheng Huang^{1,2}

¹School of Computer Science and Technology, University of Science and Technology of China, China

²Suzhou Institute for Advanced Research, University of Science and Technology of China, China

³ School of Computer Science and Engineering, Nanjing University of Science and Technology, China

INFOCOM 2025



Introduction

为解决DNN纯云端推理（需传输端侧数据，存在隐私风险且传输成本高）或纯端侧推理（资源有限，无法承担复杂推理任务）所存在的问题，出现了云端协同推理的分布式方案，其主要目标为①降低推理延迟与②增大系统吞吐量。但是由于复杂的模型结构以及动态网络条件，在流水线并行的过程中会产生大量气泡（空闲等待时间）。

现有的协同推理方法多针对单一问题（如复杂模型划分策略、并行处理），未考虑到对于气泡问题的优化。且当前方案通常是针对静态条件设计的，在动态的网络环境下往往会有明显的性能下降。

Introduction

本文提出了一种全新的协同推理框架COACH，其主要贡献有：

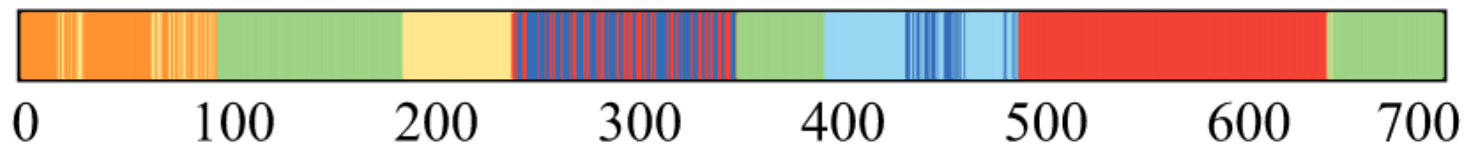
1. 第一个整合离线和在线组件的框架，专为最小化流水线气泡来优化协同推理；
2. 离线组件部分使用一种新的递归分治算法来联合确定最优的模型划分和传输量化策略；
3. 在线组件实现了自适应量化调整和上下文感知的缓存策略，根据中间数据与缓存中语义中心的相关性来调整量化精度。
4. 在保持相当精度的情况下，COACH在多种场景下实现了2.1倍推理加速与2.5倍吞吐量升。

Background

理论依据:

在协同推理任务中，数据往往会表现出重复模式。文中使用了ResNet101模型在UCF101视频数据集上进行实验，并利用t-SNE技术将数据特征可视化为一颜色线，对跨帧的中间数据进行周期分析，如下图所示。

可以看出，特征在短期内的着色具有明显的一致性，表明中间数据具有较强的时间局部性。这种观察到的相关性支持了暂时缓存先前的计算结果以加速未来的协同推理过程的可行性。



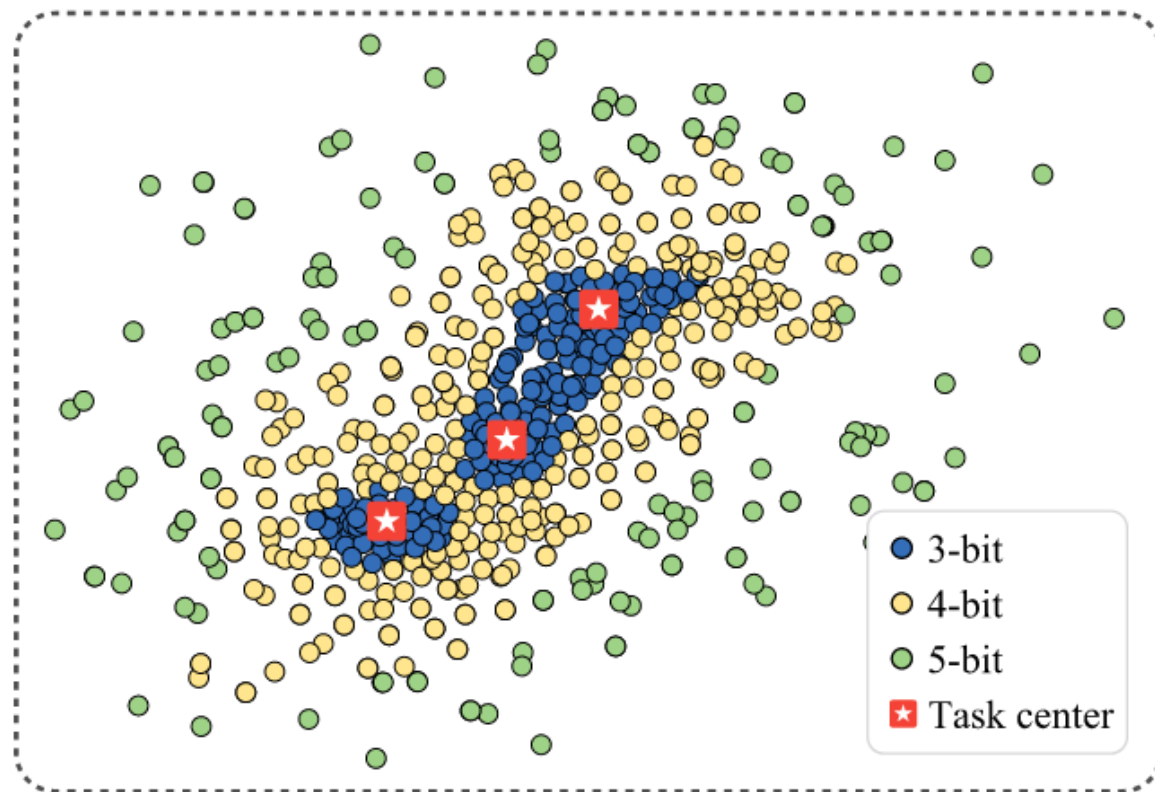
(a) Time locality visualization as a color line, in which similar colorations denote similar features.

Background

理论依据:

同时,在该实验中还将数据特征可视化作为2D散点图(右图),其中不同颜色表示不同的量化精度。可以看出3-bit的量化精度下散点几乎聚集在任务中心附近,而5-bit的散点则更加分散,这样的情况表明对数据量化精度的选择与其与任务中心的距离有关(即具有**空间局部性**),而不应是固定精度。

这为框架的“自适应量化调整”的实现提供了理论依据。



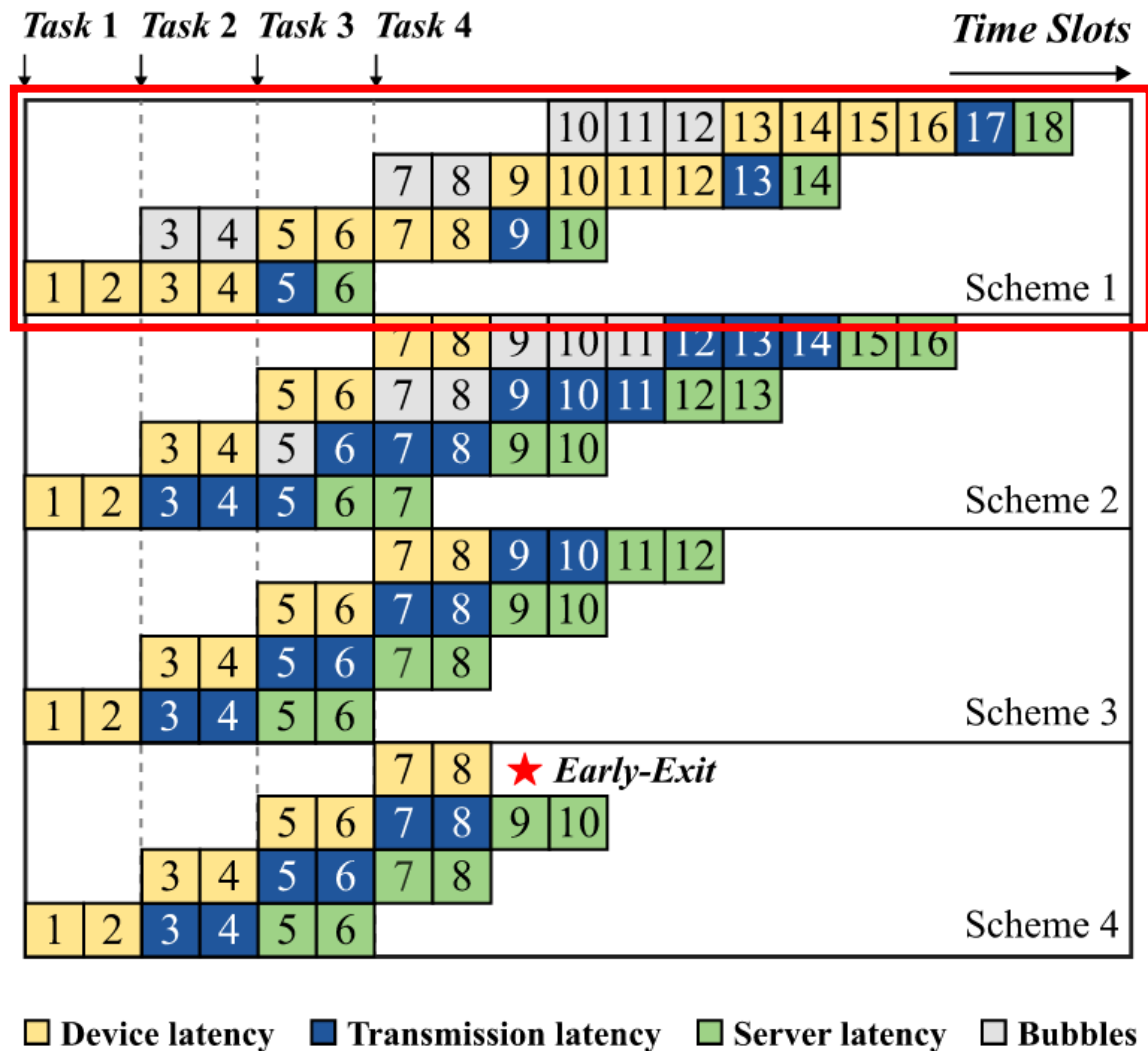
(b) Spatial locality visualization as 2D points, color-coded by optimal quantization precision.

Background

优化策略效果分析：

右图展示了三阶段(端侧-传输-云端)的协同推理过程，其中每2个时间单位依次到达4个推理任务。

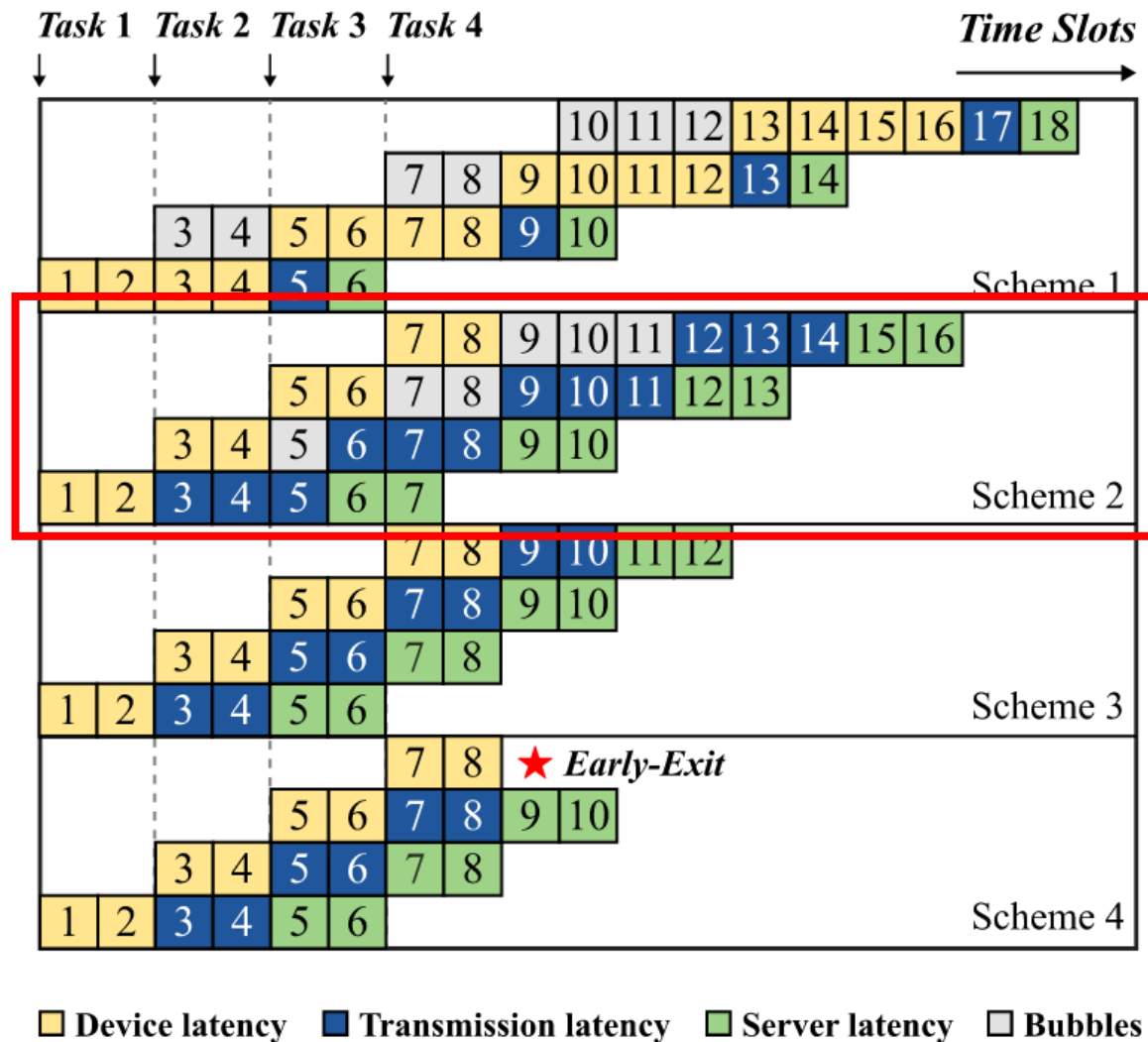
其中，方案1仅旨在最小化每个任务的延迟（六个连续时间单位执行完成），虽然已经采用了流水线并行，但并没有充分优化资源利用率，导致流水线中出现了大量气泡。



Background

优化策略效果分析:

方案2则优化了原有的模型分区策略以平衡“端侧计算-传输-云端计算”三阶段的时间。虽然使单个推理任务的延迟增至了7个时间单位，但流水线中的气泡明显减少并且提高了系统效率（流水线执行的最大阶段从4个时间单位减少到三个时间单位）。

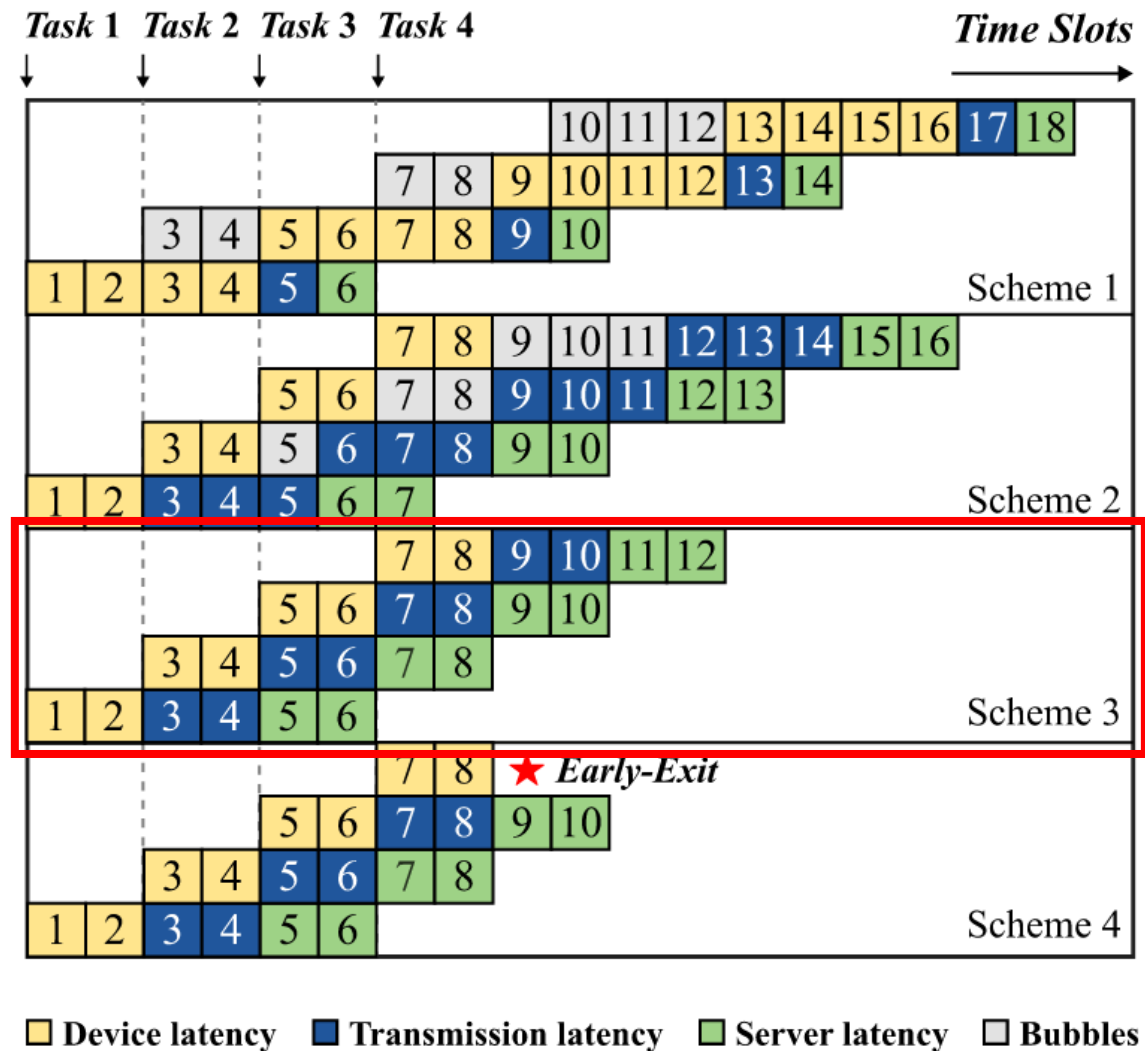


Background

优化策略效果分析：

方案3在方案2的基础之上结合先前发现的中间数据的空间局部性，又引入了 **自适应量化调整**（实时调整中间数据的量化精度）。

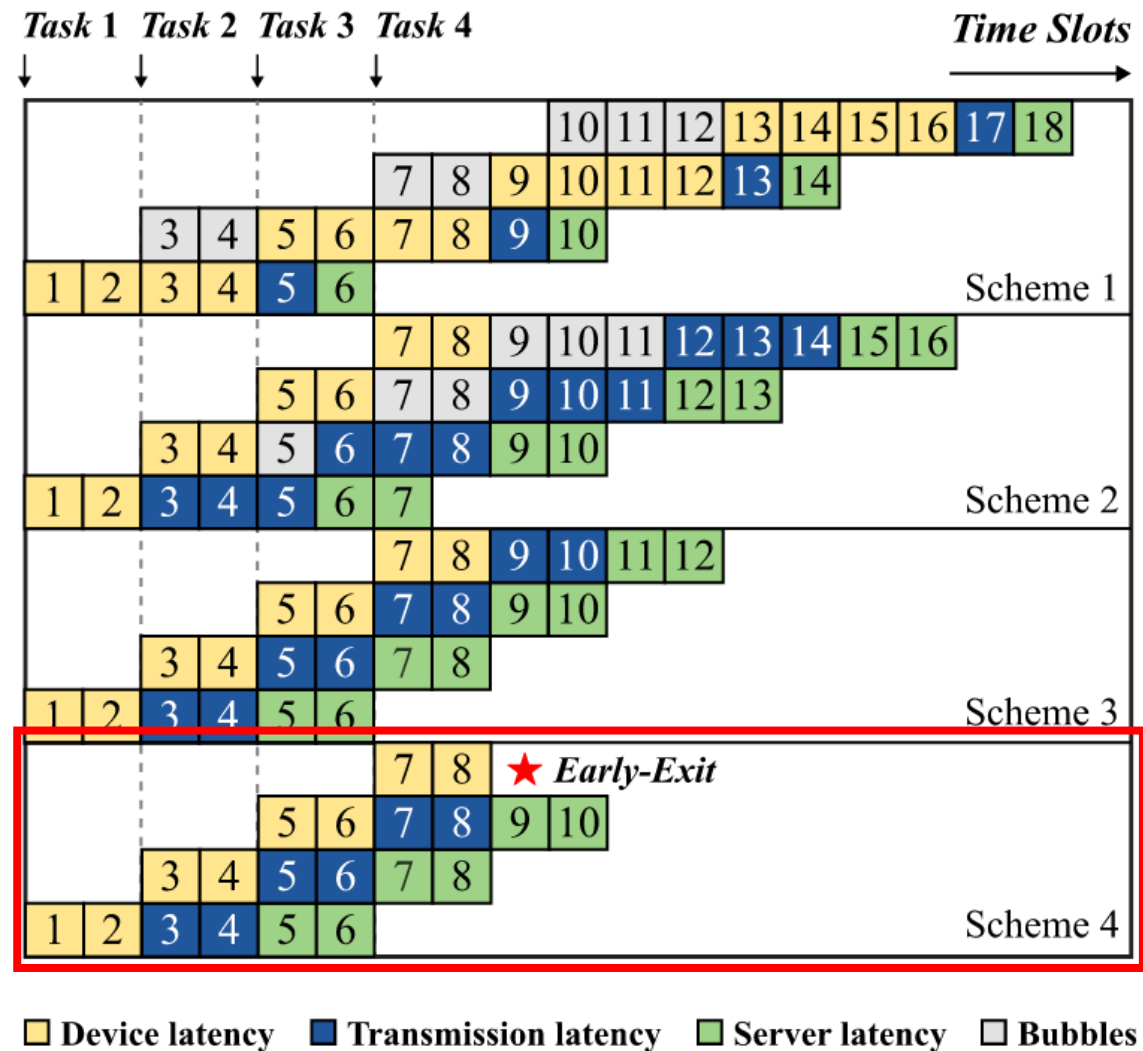
发现流水线中的气泡显著减少并且数据传输延迟降低，总效率提高。



Background

优化策略效果分析：

方案4则又在方案3的基础上基于中间数据的时间局部性又继续加入了提前退出策略（对特征与历史缓存匹配的任务，直接在端侧生成结果，无需传输至云端），进一步简化了流水线流程。

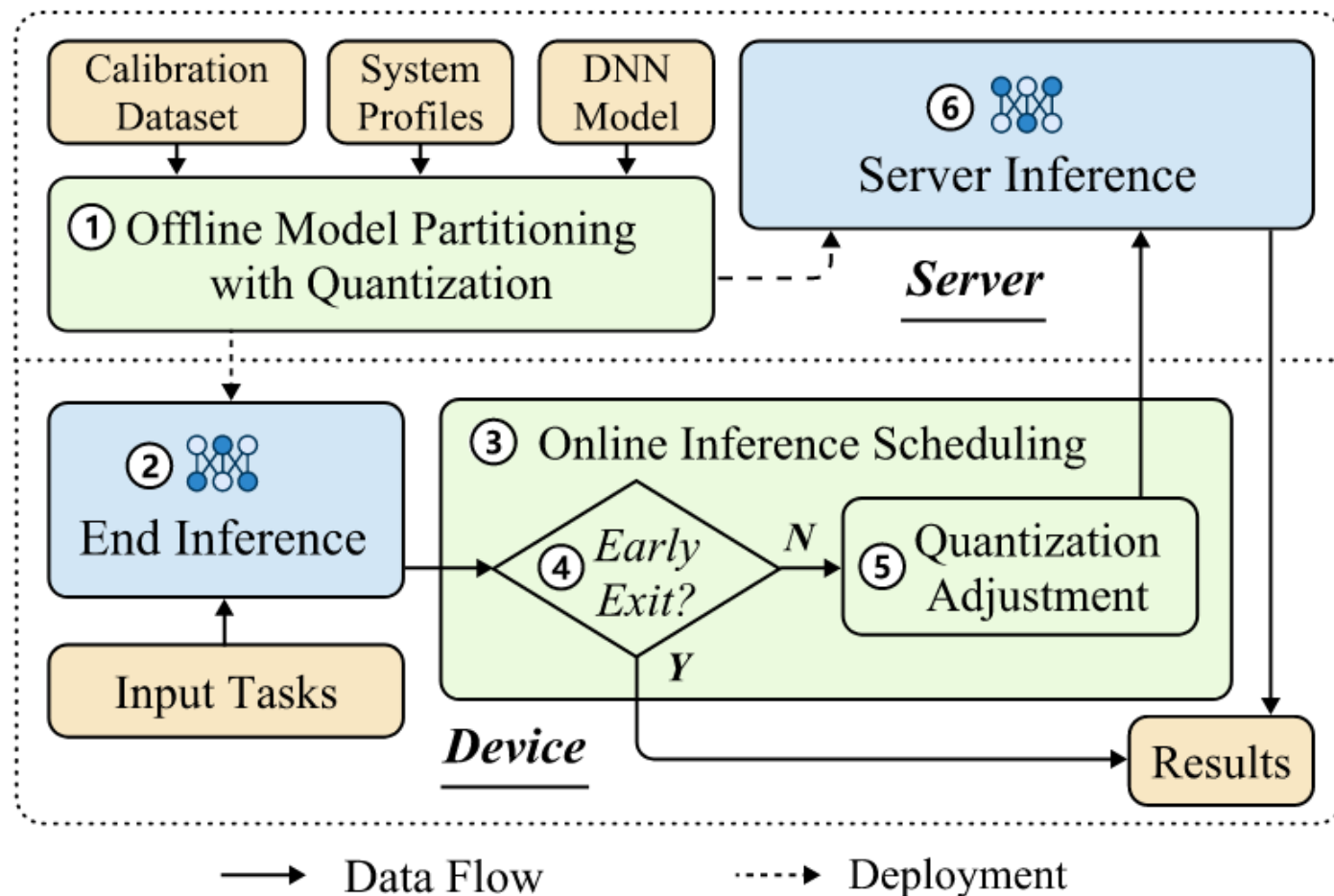


算法设计

系统概述

右图展示了COACH的完整推理工作流程，首先它从一个离线组件开始（①），在推理过程前一次性执行完成，确定模型分区方案与初始化量化精度并初始化在线组件。

在推理过程中，输入任务首先通过终端设备上的划分模型进行初步处理（②），产生中间数据。

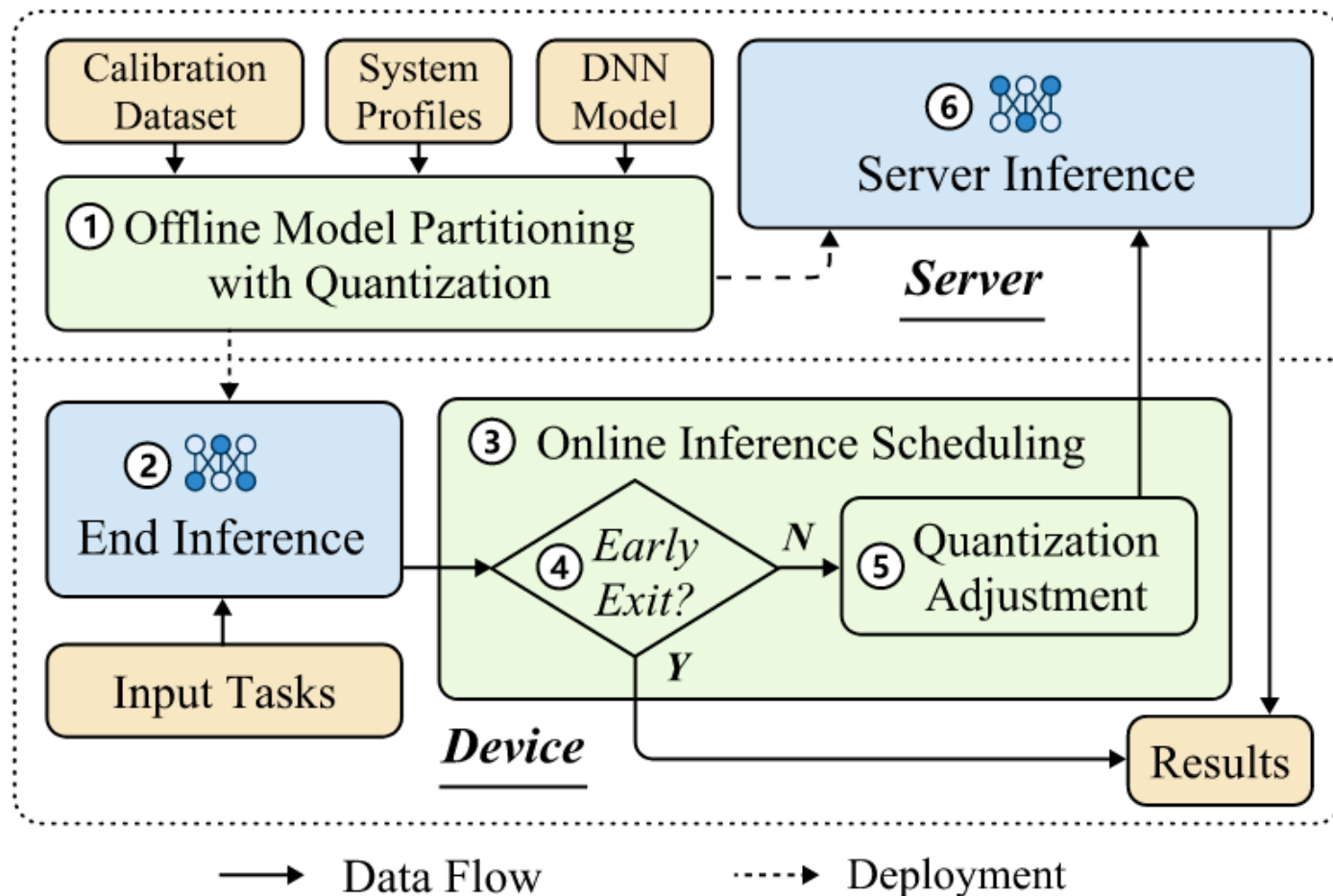


算法设计

系统概述

随后，**在线调度组件** (③) 评估中间数据的任务特定量化需求，便于进行**自适应量化调整**。

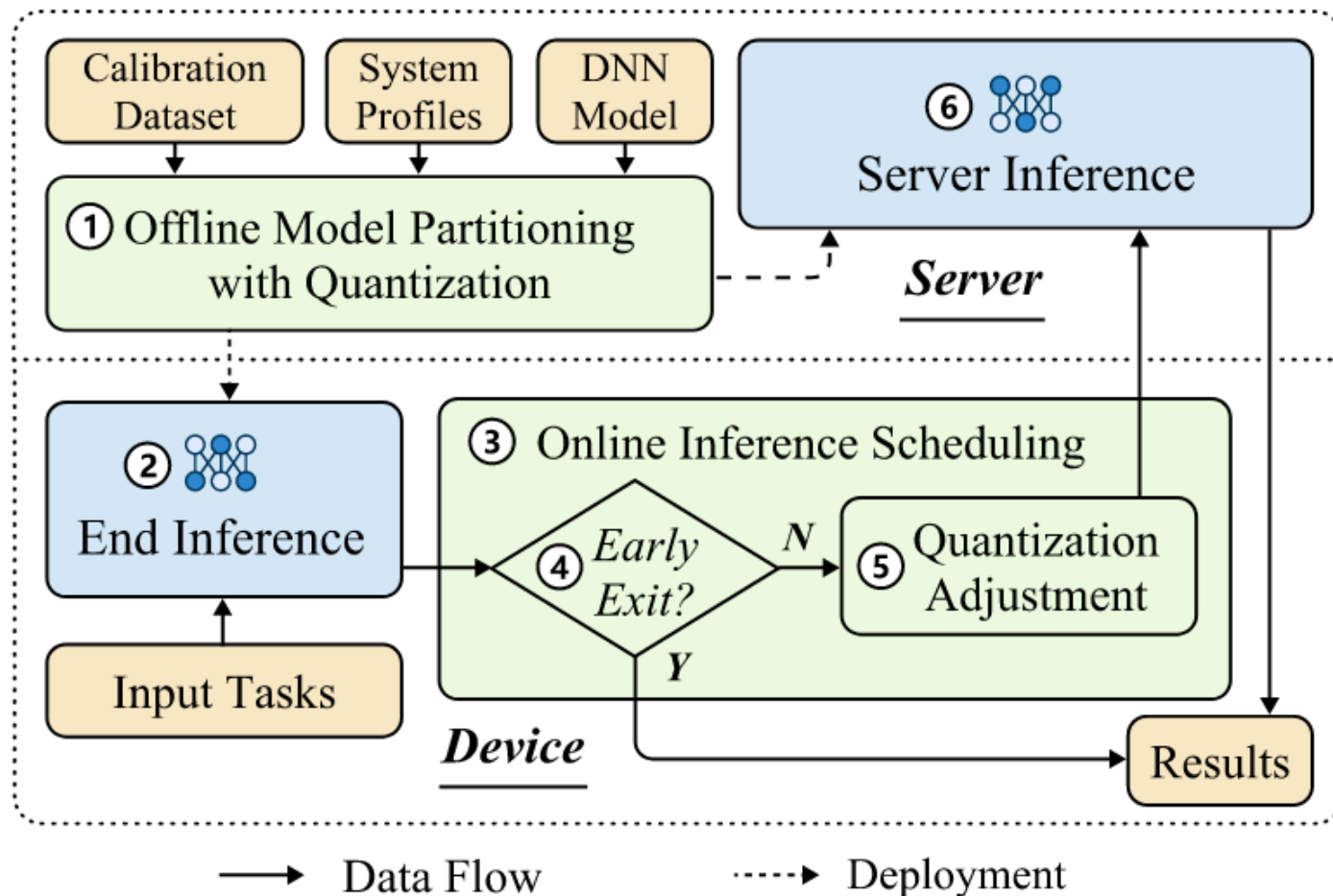
一旦量化调整满足**提前退出条件** (④)，程序将会及时生成结果。否则，在将数据传输至云服务器之前，在线组件会根据网络带宽与数据特征，自适应调整量化精度 (⑤)，压缩中间数据后传至云端。



算法设计

系统概述

最终，云服务器在接收到数据后执行剩余计算（⑥），并将推理结果返回给终端设备，完成推理任务。



算法设计

离线组件：模型分区与量化优化

为了优化传输效率，COACH利用均匀仿射量化（UAQ）技术，在保证精度的同时，高效地压缩了中间数据，且允许云服务器随后对数据进行去量化处理并继续推理过程。其中最优的量化精度是通过二分搜索，基于量化精度与推理精度之间的相关性来确定的。

精度必须满足约束：

$$|Acc(v_i) - Acc(Q(v_i))| \leq \epsilon \quad (1)$$

$Q(v_i)$ 表示层 v_i 的量化方法； $Acc(\cdot)$ 表示推理精度；

ϵ 为预设的最大允许精度损失阈值，本文设置为0.5。

算法设计

离线组件：模型分区与量化优化

流水线中三个阶段的延迟定义为：

$$T_e = \sum_{v_i \in V_e} t_i^e, \quad T_t = \sum_{v_i \in V_p} Q(t_i^t), \quad T_c = \sum_{v_i \in V_c} t_i^c, \quad (2)$$

V_e 和 V_c 分别表示在终端设备和云服务器上执行的层集合， V_p 表示分区层集合。

T_e 、 T_t 、 T_c 分别表示端侧计算总延迟、传输总延迟以及云端计算总延迟，并且其需要满足约束：

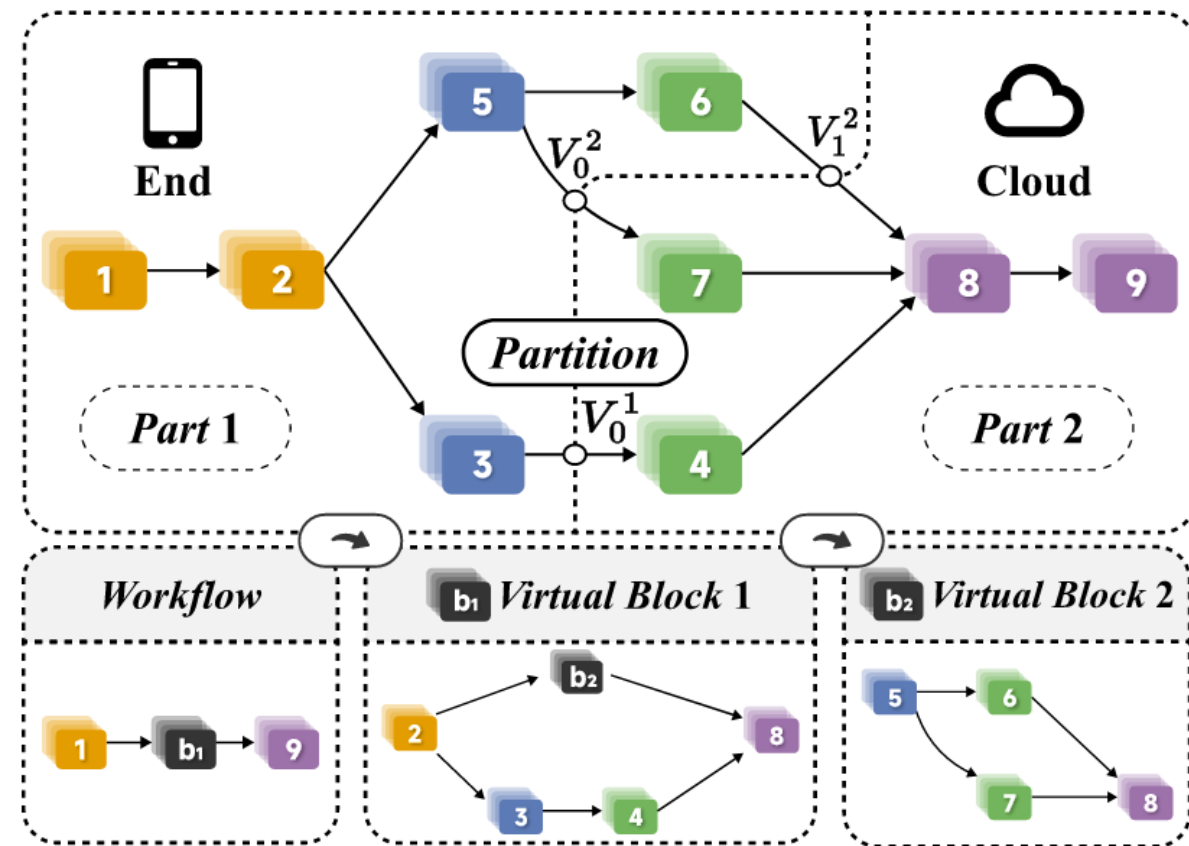
$$T_e + T_t + T_c \leq T_{max}. \quad (3)$$

算法设计

离线组件：模型分区与量化优化

在DAG结构下，通过层并行执行可以实现非相互依赖层的同时处理。例如在右图中，在第3层终端设备计算完成后，后续终端设备(第5和6层)的计算与中间数据 V_0^1 的传输可并行执行；在 V_0^1 传输完成后，第4层的云计算与 V_0^2 的传输可以并行执行。

但是在 V_0^2 的传输期间，第7层云端计算必须等待传输完成才能开始执行，这就产生了额外的气泡。



算法设计

离线组件：模型分区与量化优化

通过分析层依赖关系，考虑传输并行时间与云端并行时间 T_t^p 与 T_c^p 要满足以下约束：

$$T_t^p + T_c^p \leq \max\{T_e, T_t, T_c\}. \quad (4)$$

即保证并行总耗时不能超过瓶颈，否则会产生新的气泡。

算法设计

离线组件：模型分区与量化优化

为了量化端云协同中的两类核心气泡：计算气泡 ($B_c(V_p)$) 与传输气泡 ($B_t(V_p)$)，对其做了如下定义：

$$\begin{cases} B_c(V_p) &= |T_e - T_c|, \\ B_t(V_p) &= |T_t - \max\{T_e, T_t - T_t^p, T_c - T_c^p\}|. \end{cases} \quad (5)$$

即：

计算气泡 $B_c(V_p)$ 为端侧计算与云端计算因耗时差异产生的空闲时间；

传输气泡 $B_t(V_p)$ 为总传输延迟 T_t 与端侧总计算延迟 (T_e)、传输并行优化后的有效传输耗时 ($T_t - T_t^p$)、云端并行优化后的有效云端耗时 ($T_c - T_c^p$) 三者中最大值的绝对差值。

算法设计

离线组件：模型分区与量化优化

离线组件的目标是通过最小化气泡和延迟，确定一个最优的分区和量化策略 $\mathbf{V}^*$ ，以提高整个流水线效率。其优化问题描述如下：

$$V^* = \underset{V_p}{\operatorname{argmin}} \{B_c(V_p) + B_t(V_p) + \max\{T_e, T_t, T_c\}\}$$

$$s.t. \quad (1), (3), (4).$$

$$|Acc(v_i) - Acc(Q(v_i))| \leq \epsilon \quad (1)$$

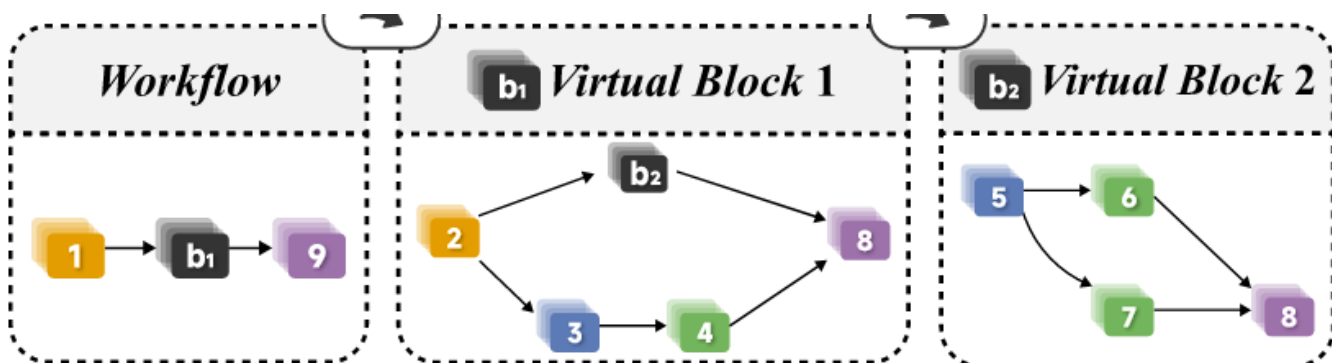
$$T_e + T_t + T_c \leq T_{max}. \quad (3)$$

$$T_t^p + T_c^p \leq \max\{T_e, T_t, T_c\}. \quad (4)$$

算法设计

离线组件：模型分区与量化优化

由于随着模型复杂度的增长，潜在的模型分区搜索空间会呈指数型增长。所以本文针对DAG拓扑模型划分的复杂性，提出了一种全新的递归分治算法，通过概念上将并行层聚类成虚拟块，并将优化重点转移到更简单的链式拓扑结构上来实现。



1 Offline Component:

- 2 Evaluate layer computation and transmission costs in model G with system profiles P .
- 3 Cluster parallel layers into virtual blocks within G .
- 4 Determine chain flow $B_g = \{b_1, b_2, \dots, b_n\}$ in G .
- 5 Initialize search space $S = \{B_g\}$ and offline strategy $V_p = \emptyset$.
- 6 **for** each chain flow B in S **do**
 - 7 Initial offline strategy $V_B = \emptyset$ for B .
 - 8 **for** each block b in B **do**
 - 9 Determine quantization precision for block b .
 - 10 Calculate bubble functions $B_c(V_B), B_t(V_B)$.
 - 11 Assess V_B performance in pipeline execution.
 - 12 Update the optimal offline strategy V_B for chain flow B .
 - 13 **if** b is a virtual block **then**
 - 14 Append additional chain flows from b to S .
 - 15 Update V_p with V_B .
- 16 Deploy the optimal strategy V^* to device and server.

算法设计

离线组件：模型分区与量化优化

离线组件开始评估模型G中每一层的计算和传输成本

在模型G中将并行层聚成虚拟块，将这些块组织成顺序链流B并将其合并到搜索空间S中

该算法通过迭代地评估搜索空间S内的每条链流，旨在确定相应的最佳分区和量化策略。对于搜索空间S中的每条链流B，算法首先利用二分搜索精确地确定推理数据流的合适量化精度。随后，通过识别层间的依存关系分析并行执行时间 T_t^p 与 T_c^p ，并同时计算计算和传输气泡函数 $B_c(V_p)$ 与 $B_t(V_p)$ ，从而评估最优策略。

1 Offline Component:

- 2 Evaluate layer computation and transmission costs in model G with system profiles P .
- 3 Cluster parallel layers into virtual blocks within G .
- 4 Determine chain flow $B_g = \{b_1, b_2, \dots, b_n\}$ in G .
- 5 Initialize search space $S = \{B_g\}$ and offline strategy $V_p = \emptyset$.
- 6 **for** each chain flow B in S **do**
 - 7 Initial offline strategy $V_B = \emptyset$ for B .
 - 8 **for** each block b in B **do**
 - 9 Determine quantization precision for block b .
 - 10 Calculate bubble functions $B_c(V_B), B_t(V_B)$.
 - 11 Assess V_B performance in pipeline execution.
 - 12 Update the optimal offline strategy V_B for chain flow B .
 - 13 **if** b is a virtual block **then**
 - 14 Append additional chain flows from b to S .
 - 15 Update V_p with V_B .
 - 16 Deploy the optimal strategy V^* to device and server.

算法设计

离线组件：模型分区与量化优化

在链流 B 中，算法依次检查每个虚拟块，将不适合该策略的虚拟块排除在进一步分析之外。反之，将合适的虚拟块递归地集成到搜索空间 S 中进行优化。对 B 进行综合评价后，用 V_B 对离线策略 V^* 进行更新。

最终将最优策略部署到设备和服务器上。

1 Offline Component:

- 2 Evaluate layer computation and transmission costs in model G with system profiles P .
- 3 Cluster parallel layers into virtual blocks within G .
- 4 Determine chain flow $B_g = \{b_1, b_2, \dots, b_n\}$ in G .
- 5 Initialize search space $S = \{B_g\}$ and offline strategy $V_p = \emptyset$.
- 6 **for** each chain flow B in S **do**
 - 7 Initial offline strategy $V_B = \emptyset$ for B .
 - 8 **for** each block b in B **do**
 - 9 Determine quantization precision for block b .
 - 10 Calculate bubble functions $B_c(V_B), B_t(V_B)$.
 - 11 Assess V_B performance in pipeline execution.
 - 12 Update the optimal offline strategy V_B for chain flow B .
 - 13 **if** b is a virtual block **then**
 - 14 Append additional chain flows from b to S .
 - 15 Update V_p with V_B .
- 16 Deploy the optimal strategy V^* to device and server.

算法设计

在线组件：推理调度与动态适配

虽然离线组件部分已经在模型划分和量化策略方面做了优化，一定程度上减少了流水线中的气泡，提高了系统效率与吞吐量。但是在实际的推理场景中会出现网络波动导致传输延迟增加，产生大量气泡；并且由于不同推理任务的中间数据与“语义中心”的相关性不同，离线组件固定的量化精度无法适配所有任务。所以在在线组件部分提出了两个核心策略：

1. 自适应量化调整策略；
2. 提前退出策略。

算法设计

在线组件

为了有效管理大量的中间数，**COACH**使用了全局平均池化（GPA）函数，它的输入是维度为 $[C \times H \times W]$ 的中间数据，输出是维度为C的任务特征F。其中C表示通道数，H与W表示空间维数。

此外，为了做出基于相似度的量化调整决策，缓存机制为每个每个标签维护了一个语义中心，中心记为 $\mathbf{T}_c = \{T_j^c\}$ ，其中标签 $j \in \{0, \dots, n\}$ 。

通过评估中间数据与缓存中标签语义中心的相关性，可以动态调整量化精度，最小化管道气泡。

算法设计

在线组件

这些标签语义中心最初随着校准数据集D进行预热，在推理过程中会随着任务特征逐步更新。它通过不断融入新的任务特征，保持对当前条件的真实反映，从而减少潜在的数据偏差。标签语义中心的更新如下：

$$T_j^c = \frac{m_j T_j^c + F_j}{m_j + 1}$$

其中 m_j 表示任务的数量； T_j^c 表示语义中心； F_j 是与标签j相关的当前任务特征。

算法设计

在线组件

为了评估相似性，本文通过余弦相似度将任务特征与标签语义中心进行比较，并为每个标签建立相似度 $\mathbf{T} = \{t_j\}$ 。标签j的相似度形式化为：

$$t_j = \xi(F, T_j^c) \in [0, 1]$$

其中函数 $\xi(\cdot) \in [0, 1]$ 为余弦相似度函数。 t_j 量化了任务特征F与标签语义中心 T_j^c 的相似程度， $t_j=1$ 表示完全相似， $t_j=0$ 表示完全不相似。

算法设计

在线组件

此外，为了评估特定任务中间数据的量化精度要求，还引入了任务可分性概念,用于量化当前任务特征与标签语义中心的区分度,记为：

$$S = \|\mathbf{T}\|_2 \cdot (t_H - t_{SH}) \frac{t_H}{t_{SH}}, \quad (9)$$

其中 t_H 与 t_{SH} 分别表示相似度集合 T 中的最高和次高相似度；

$\|\mathbf{T}\|_2$ 为相似度集合 T 的L2范数（即所有 t_j 的平方和开根）。

若 S 越大，则表示中间数据与标签语义中心之间的相关性越明显，表明推理结果对目标标签越可靠。

算法设计

在线组件：提前退出策略

在离线组件部分会利用校准数据集D初步建立了早退出阈值 S_{ext} , 确保精度损失低于0.5%。当任务可分性S超过缓存阈值 S_{ext} 时, 满足提前退出条件, 直接输出结果, 否则继续执行。提前退出结果R的决策框架封装为以下方程:

$$R = \underset{j \in \{0, \dots, n\}}{\operatorname{argmax}} \{t_j \in \mathbf{T}\}, \quad (10)$$

其中, 通过识别T内所有标签之间的最高相似度来确定结果。提前退出策略确保决策基于最相关和最相似的数据特征进行预测, 从而在保持推断准确性的同时优化运行效率。

算法设计

在线组件：自适应量化策略

此外，在初始阶段还会初步建立一个量化精度阈值 S_{adj} 。在动态的网络环境中，量化调整对于流水线的稳定执行是很重要的，这种调整依赖于一个前提，即更大的任务可分性 S 会使得量化方法更加激进（量化精度更低）。通过比较任务可分性 S 与量化调整阈值 S_{adj} ，确定当前任务的量化精度 Q_r 。利用该精度，再通过最小化冒泡函数对量化调整 Q_c 进行实时决策：

$$Q_c = \operatorname{argmin}_{Q_c \geq Q_r} \{|T'_t - \max\{T_e, T'_t, T_c\}|\}, \quad (11)$$

其中 T'_t 表示传输时间（与量化精度 Q_c 与实时带宽 B 相关）。

算法设计

在线组件

- 17 **Online Component:**
- 18 Initialize semantic centers $\mathbf{T}_c$ by calibration dataset D .
- 19 Evaluate early-exit threshold S_{ext} and quantization thresholds $\mathbf{S}_{adj}$.
- 20 **while** *Receive task feature F* **do**
- 21 Get similarity degrees $\mathbf{T}$ and task separability S .
- 22 **if** $S > S_{ext}$ **then**
- 23 Update $T_j^c \in \mathbf{T}_c$ by Eq. (7).
- 24 Get inference result R from $\mathbf{T}_c$ by Eq. (10).
- 25 Continue.
- 26 Get real-time network bandwidth and estimate quantization requirement Q_r with $\mathbf{S}_{adj}$.
- 27 Quantize F with Q_c by Eq. (11)

实验设置

云服务器： 设置一个高性能的AMAX深度学习工作站作为云服务器，搭载4个NVIDIA A6000 GPU；

终端设备： 一个Nvidia Jetson Xavier NX和一个Nvidia Jetson TX2。其中NX模型搭载了6核的NVIDIA Carmel ARMv8。2个CPU和一个384核的NVIDIA Volta GPU，辅以8GB的RAM；TX2具有4核ARM Cortex-A57 CPU和256核NVIDIA Pascal GPU，并具有8GB的RAM；

网络条件： 一个5GHz的WiFi路由器，模拟真实世界的网络状况；

数据集： UCF101视频数据集、ImageNet-100图像数据集数据集；

模型： ResNet101模型、VGG16模型。

Baselines:Neurosurgeon(NS)、DADS、SPINN、JPS

实验设置

下表为COACH框架与四种baselines在不同端设备与不同模型下的平均推理延迟；，
可以看到COACH框架在每种情况下的推理延迟都是最低的。

TABLE I: Average Inference Latency (ms) for COACH and baselines.

	Resnet101		VGG16	
	NX	TX2	NX	TX2
<i>NS</i>	45.16	62.67	29.52	52.73
<i>DADS</i>	38.11	49.34	24.38	39.50
<i>SPINN</i>	22.04	30.10	14.03	19.97
<i>JPS</i>	20.78	28.31	12.52	18.13
<i>COACH</i>	15.63	19.11	9.71	13.37

实验设置

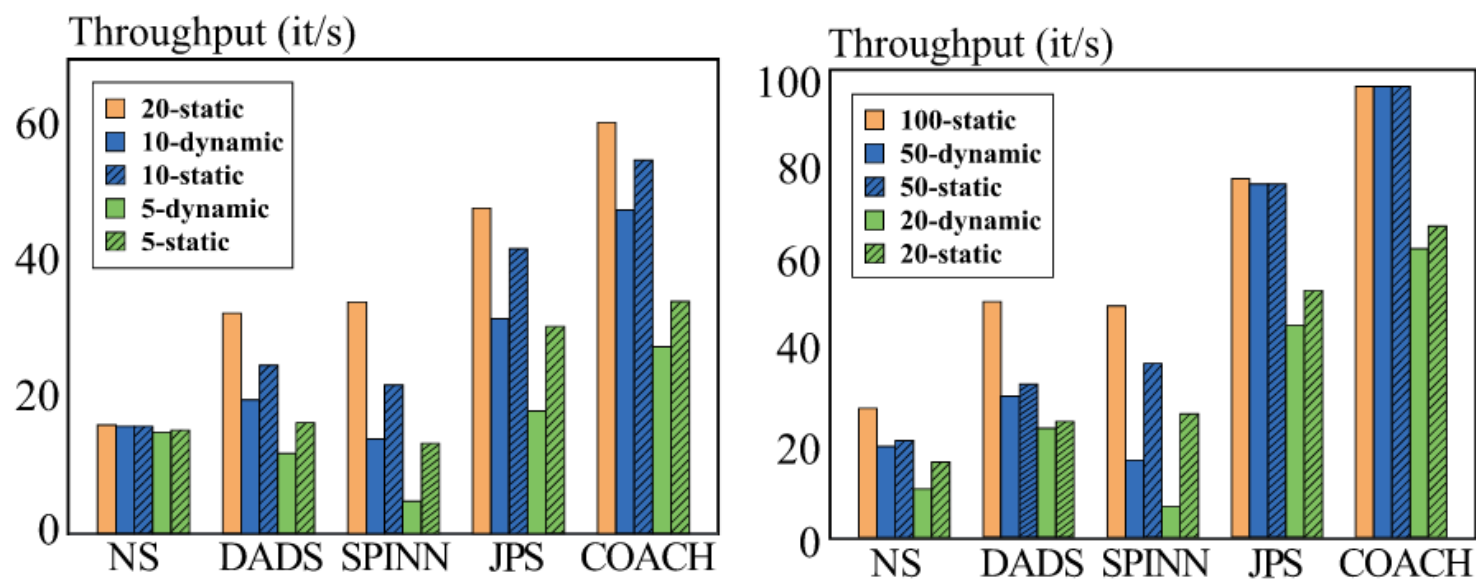
下表展示了COACH框架在不同的数据相关性水下在两种模型上的性能表现。

随着数据相关性的提高，模型的早退出率也随之提高，推理延迟与传输成本均显著下降。

	早退出率	推理延迟	传输成本			
	↑	↑	↑			
	Resnet101			VGG16		
	EER	Lat.	Trans.	EER	Lat.	Trans.
<i>NoAdjust</i>	-	13.63	121.0	-	12.02	98.0
<i>Low</i>	11.86%	12.19	102.9	19.77%	10.64	75.3
<i>Medium</i>	37.41%	9.68	80.2	52.69%	7.71	44.1
<i>High</i>	65.38%	6.13	40.3	76.58%	5.77	20.6

实验设置

动态的网络环境下，COACH框架在吞吐量方面也有更优的表现。



(a) Initial at 20Mbps, reduced to 10Mbps and 5Mbps

(b) Initial at 100Mbps, reduced to 50Mbps and 20Mbps

Fig. 5: Adaptability of COACH and baselines in dynamic network conditions.

实验设置

不同场景下COACH与Baselines的系统延迟情况对比

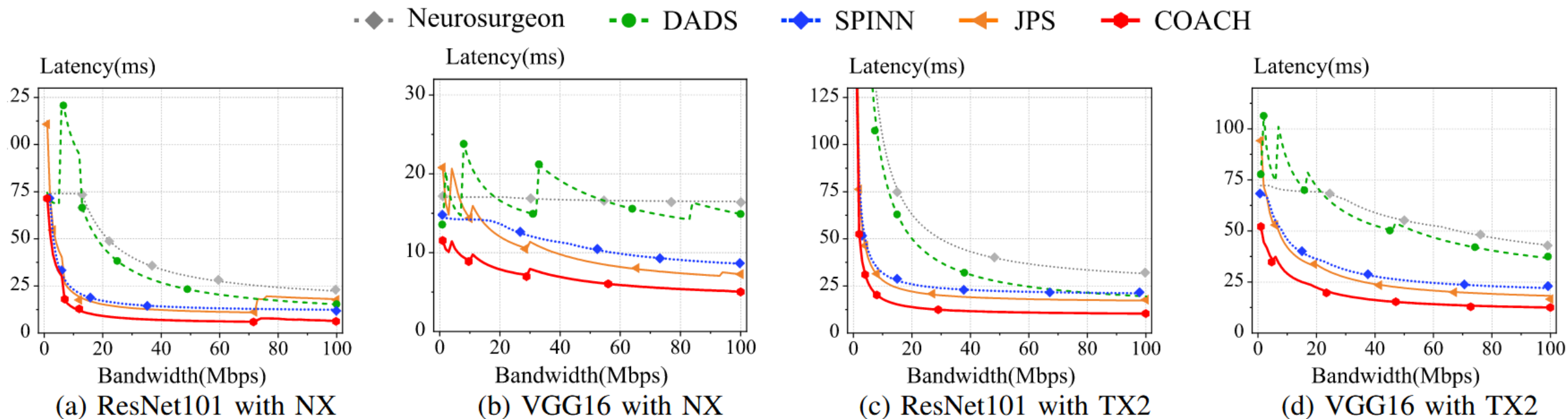


Fig. 6: Latency of COACH and baselines on Resnet101 and VGG16 in different scenarios.

实验设置

不同场景下COACH与Baselines的系统吞吐量情况对比

-◆- Neurosurgeon -●- DADS -◆- SPINN -▶- JPS -◆- COACH

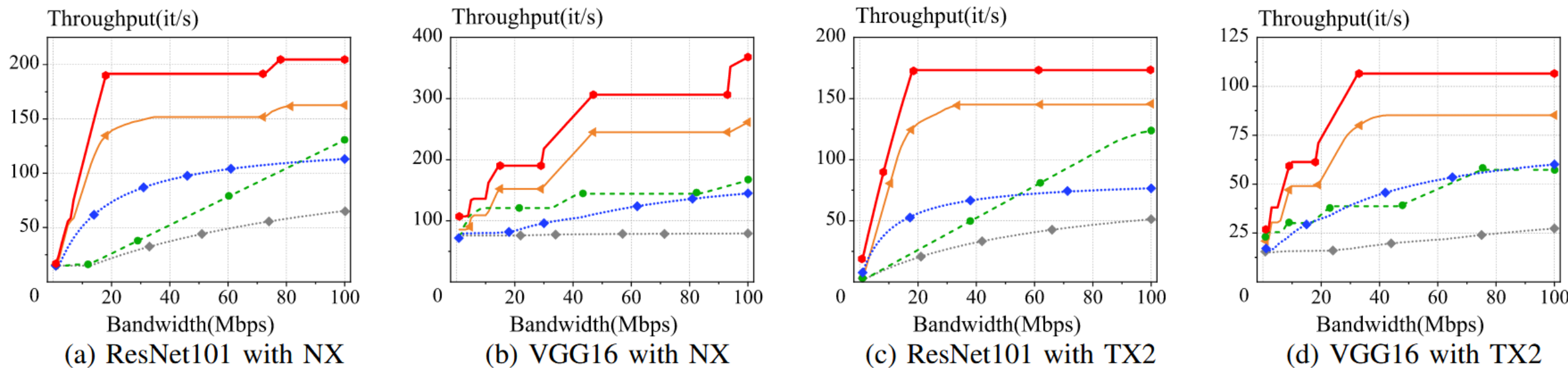


Fig. 7: Throughput of COACH and baselines on Resnet101 and VGG16 in different scenarios.



汇报完毕
恳请指正